

ANKARA UNIVERSITY
FACULTY OF ENGINEERING
DEPARTMENT OF COMPUTER ENGINEERING



BLM4522 Ağ Tabanlı Paralel Tanıtım Sistemleri

Proje 3: Veritabanı Güvenliği ve Erişim Kontrolü

Tuğrul Özgün

22290082

Git link: <https://github.com/Forcipus/Paralel-Sistemler>

Video link: <https://drive.google.com/drive/folders/1Y5rMBLKE9o8Q7bGbx6K-PDCzmVk92KUc?usp=sharing>

İÇİNDEKİLER

1. GİRİŞ	2
1.1. Kapsam ve Metodoloji	
2. ERİŞİM YÖNETİMİ	2
2.1. SQL Server Authentication ile Kimlik Doğrulama	
2.2. Kullanıcı Yetkilendirme (User Mapping) ve Rol Atamaları	
2.3. Windows Authentication (Entegre Güvenlik)	
3. VERİ ŞİFRELEME (DATA ENCRYPTION)	4
3.1. Transparent Data Encryption (TDE - Şeffaf Veri Şifreleme)	
3.2. Always Encrypted (Her Zaman Şifreli)	
3.3. Column-Level Encryption (Hücre Bazlı Şifreleme)	
3.4. Dynamic Data Masking (DDM) ile Veri Maskeleyme	
4. SQL INJECTION TESTLERİ VE SAVUNMA STRATEJİLERİ	6
4.1. SQL Injection Nedir? (Zafiyetin Mantığı)	
4.2. Adım Adım SQL Injection Testi (Simülasyon)	
4.3. SQL Injection'dan Korunma Yöntemleri	
4.4. Uygulamalı Savunma Analizi	
5. AUDIT LOGLARI (DENETİM İZLERİ)	8
5.1. Denetim Mekanizmasının Amacı	
5.2. Sunucu Düzeyinde Denetim (Server Audit) Oluşturma	
5.3. Veritabanı Düzeyinde Denetim (Database Audit Specification)	
5.4. Denetim Kayıtlarının İzlenmesi ve Analiz	
6. SONUÇ VE DEĞERLENDİRME	9

1. GİRİŞ

Bilgi güvenliğinin temel taşı oluşturarak veritabanı katmanı; gizlilik, bütünlük ve erişilebilirlik prensipleri çerçevesinde korunmalıdır. Veritabanları, sadece operasyonel verileri değil, kişisel veriler ve ticari sırlar gibi kritik varlıkları barındırdığı için siber saldırıların birincil hedefidir.

Bu projenin temel amacı, **Microsoft SQL Server** üzerinde "Çok Katmanlı Savunma" (Defense-in-Depth) stratejilerini **WideWorldImporters** veri seti üzerinden uygulamalı olarak simüle etmektir. Çalışma kapsamında kullanıcı yetkilendirme süreçleri, veri şifreleme yöntemleri, SQL Injection saldırılarına karşı savunma mekanizmaları ve denetim logları dökümanite edilmiştir.

1.1. Kapsam ve Metodoloji

Proje süresince veri güvenliğini uçtan uca sağlamak adına aşağıdaki teknik süreçler gerçekleştirilmiştir:

- **Erişim Yönetimi:** Windows ve SQL Server Authentication yöntemleri kullanılarak kimlik doğrulama süreçleri yapılandırılmış, "En Az Yetki" (Least Privilege) prensibi ile kullanıcı rolleri atanmıştır.
- **Veri Şifreleme (Encryption at Rest):** Veritabanı dosyalarının fiziksel güvenliği için Transparent Data Encryption (TDE) ve hassas veriler için gelişmiş şifreleme yöntemleri ele alınmıştır.
- **Zafiyet Testleri ve Savunma:** SQL Injection saldırı vektörleri simüle edilerek, parametrelili sorgu ve Stored Procedure kullanımı ile bu saldırıların nasıl engellendiği teknik olarak gösterilmiştir.
- **İzlenebilirlik ve Denetim (Auditing):** SQL Server Audit özellikleri aktifleştirilerek, veritabanı üzerindeki tüm kullanıcı hareketlerinin kayıt altına alınması ve hesap verebilirlik mekanizması kurulmuştur.

Bu rapor, söz konusu güvenlik protokollerinin teknik konfigürasyonlarını, uygulama sonuçlarını ve siber güvenlik odaklı mühendislik çözümlerini içermektedir.

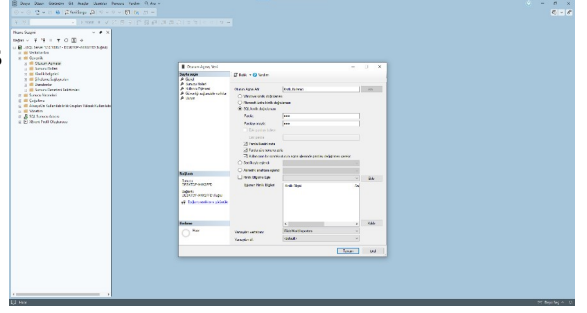
2. ERİŞİM YÖNETİMİ

Veritabanı güvenliğinin ilk katmanı olan erişim yönetimi, "Least Privilege" (En Az Yetki) prensibi doğrultusunda yapılandırılmıştır. Bu bölüm kapsamında, hem uygulama tabanlı erişimler için kullanılan **SQL Server Authentication** hem de kurumsal güvenlik standartlarına uygun **Windows Authentication** yöntemleri simüle edilmiştir.

2.1. SQL Server Authentication ile Kimlik Doğrulama

Uygulama bazlı erişim senaryolarını test etmek amacıyla, kimlik bilgilerinin doğrudan SQL Server motoru tarafından yönetildiği bir kullanıcı hesabı oluşturulmuştur.

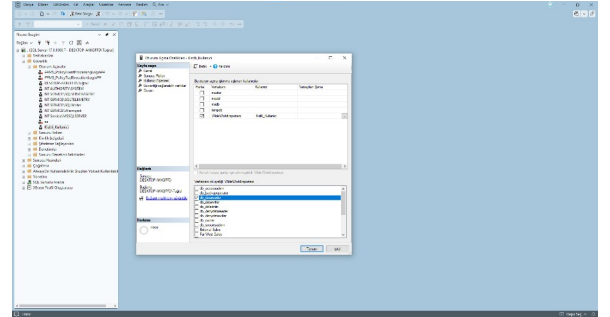
- **Bağlantı ve Erişim:** SSMS (SQL Server Management Studio) arayüzü üzerinden Windows Authentication ile sunucuya bağlanılmış ve **Security > Logins** sekmesi üzerinden yeni bir oturum açma (Login) süreci başlatılmıştır.
- **Kullanıcı Tanımlama:** "General" sekmesinde oturum açma adı olarak **Kisitli_Kullanici** belirlenmiştir.
- **Kimlik Doğrulama Türü:** "SQL Server authentication" seçeneği işaretlenerek, parolanın sunucu tarafından kontrol edilmesi sağlanmıştır.
- **Güvenlik Politikaları:** Parola politikalarını (Password Policy) zorunlu tutan "Enforce password policy" seçeneği aktif bırakılarak hesap güvenliği standartlara uygun hale getirilmiştir.
- **Varsayılan Veritabanı:** Kullanıcının sisteme girişiyle birlikte doğrudan **WideWorldImporters** bağlamında çalışması için varsayılan veritabanı ayarı yapılmıştır.



2.2. Kullanıcı Yetkilendirme (User Mapping) ve Rol Atamaları

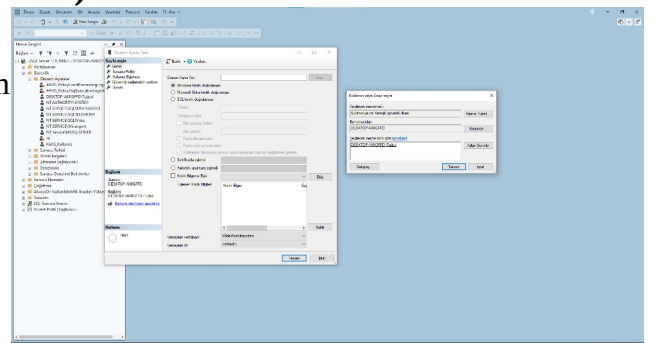
Oluşturulan Login nesnesinin veritabanı düzeyinde hangi işlemleri yapabileceği, yetki matrisi üzerinden yapılandırılmıştır.

- **Veritabanı Eşleme:** "User Mapping" sekmesi üzerinden kullanıcının **WideWorldImporters** veritabanına erişim yetkisi tanımlanmıştır.
- **Rol Ataması (Authorization):** Veri güvenliğini sağlamak ve yetkisiz değişiklikleri engellemek amacıyla kullanıcıya yalnızca aşağıdaki roller atanmıştır:
 - **public:** Tüm kullanıcılar için geçerli olan temel roller.
 - **db_datareader:** Kullanıcının tüm tabloları okumasına izin veren ancak veri silme (DELETE), güncelleme (UPDATE) veya ekleme (INSERT) yetkilerini kısıtlayan kritik güvenlik rolü.



2.3. Windows Authentication (Entegre Güvenlik)

En güvenli kimlik doğrulama yöntemi olarak kabul edilen Windows Authentication ile kullanıcının işletim sistemi oturumu üzerinden şifresiz ve güvenli bir şekilde bağlanması sağlanmıştır.



- **Sistem Entegrasyonu:** "Search..." butonu kullanılarak işletim sistemi üzerindeki aktif kullanıcı hesabı veritabanı motoruna dahil edilmiştir.
- **Doğrulama (Check Names):** "Check Names" özelliği ile Windows Active Directory veya yerel kullanıcı listesi üzerinden isim doğrulaması gerçekleştirilmiştir.
- **Kısıtlı Yetkilendirme:** Kurumsal hiyerarşiye uygun olarak, bu kullanıcı hesabı için de yalnızca db_datareader ve public rolleri tanımlanarak veri üzerinde salt okunur (read-only) erişim kısıtlaması uygulanmıştır.

3. VERİ ŞİFRELEME (DATA ENCRYPTION)

Veritabanı güvenliğinin en kritik katmanlarından biri olan veri şifreleme; verilerin hem disk üzerinde (data at rest) hem de ağ üzerinde taşınırken (data in transit) yetkisiz kişilerce okunamaz hale getirilmesini sağlar. Proje kapsamında, WideWorldImporters veritabanı üzerinde dosya, hücre ve uygulama düzeyinde farklı modern şifreleme yöntemleri uygulanmıştır.

3.1. Transparent Data Encryption (TDE - Şeffaf Veri Şifreleme)

TDE, veritabanını dosya düzeyinde (.mdf ve .bak) şifreleyerek fiziksel hırsızlıklara karşı koruma sağlar. Bu yöntemin en büyük avantajı, uygulama katmanında herhangi bir kod değişikliği gerektirmemesidir.

Uygulama Adımları: TDE'nin aktif edilmesi için SQL Server şifreleme hiyerarşisi şu adımlarla kurulmuştur:

1. **Master Key Oluşturma:** Sunucu düzeyinde tüm anahtarları koruyacak olan ana anahtar oluşturulmuştur.

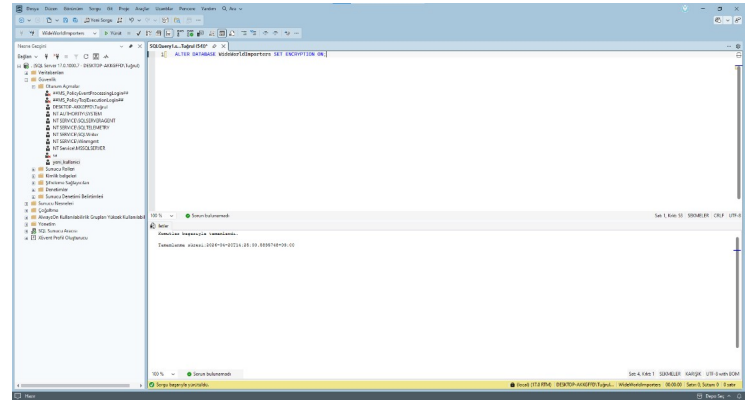
```
CREATE MASTER KEY ENCRYPTION BY
PASSWORD = 'GucluSifre123!';
```

2. **Sertifika Oluşturma:** Şifreleme anahtarını koruyacak olan sunucu sertifikası hazırlanmıştır.

```
CREATE CERTIFICATE TDE_Sertifikasi WITH SUBJECT = 'Veri Sifreleme
Sertifikasi';
```

3. **Database Encryption Key (DEK) Oluşturma:** İlgili veritabanına gidilerek AES_256 algoritması ile anahtar tanımlanmıştır.

```
USE WideWorldImporters;
CREATE DATABASE ENCRYPTION KEY
WITH ALGORITHM = AES_256
ENCRYPTION BY SERVER CERTIFICATE TDE_Sertifikasi;
```



4. Şifrelemeyi Başlatma:

```
ALTER DATABASE WideWorldImporters SET ENCRYPTION ON;
```

3.2. Always Encrypted (Her Zaman Şifreli)

TDE diskteki dosyayı korurken, veritabanı yöneticisinin (DBA) verileri görmesini engelleyemez. **Always Encrypted** teknolojisi, veriyi henüz istemci (uygulama) tarafındayken şifreleyerek SQL Server'ın bile içeriği görmesini engeller.

- **Uygulama:** SSMS üzerinde ilgili tabloya sağ tıklanarak **Encrypt Columns** sihirbazı kullanılmış, hassas sütunlar (Örn: TC Kimlik No veya Kredi Kartı) için şifreleme anahtarları oluşturulmuştur.

Always Encrypted ile veriler SQL Server belleğinde dahi şifreli tutularak, yüksek yetkili kullanıcıların (DBA) hassas verilere erişimi teknik olarak imkansız hale getirilmiştir.

3.3. Column-Level Encryption (Hücre Bazlı Şifreleme)

Tüm veritabanı yerine yalnızca belirli kolonların (Örn: CreditCardNumber) şifrlenmesi gerektiği durumlarda tercih edilen yöntemdir. Simetrik bir anahtar oluşturulur ve veriler ENCRYPTBYKEY fonksiyonu ile şifrlenerek kaydedilir. Okuma sırasında ise DECRYPTBYKEY fonksiyonu kullanılır.

- **Örnek Sorgu:**

```
OPEN SYMMETRIC KEY Kart_Anahtari DECRYPTION BY CERTIFICATE  
TDE_Sertifikasi;  
UPDATE Sales.Customers  
SET CreditCard = EncryptByKey(Key_GUID('Kart_Anahtari'), '1234-5678-  
9012');
```

3.4. Dynamic Data Masking (DDM - Dinamik Veri Maskeleye)

DDM, veriyi fiziksel olarak şifrelemez; ancak yetkisi olmayan kullanıcılara veriyi maskeli (yıldızlı) şekilde sunar. Bu, özellikle destek ekiplerinin hassas verileri görmeden işlem yapabilmesi için kritik bir güvenlik protokolüdür.

- **Uygulama:** E-posta adreslerini maskeleymek için şu komut kullanılmıştır:

```
ALTER TABLE Application.People  
ALTER COLUMN EmailAddress  
ADD MASKED WITH (FUNCTION = 'email()');
```

Kısıtlı bir kullanıcı bu kolonu sorguladığında veriyi a***@gmail.com formatında görerek veri gizliliği korunmuş olur.

4. SQL INJECTION TESTLERİ VE SAVUNMA STRATEJİLERİ

SQL Injection (SQLi), bir saldırganın uygulama formları veya URL parametreleri aracılığıyla veritabanı motoruna kötü niyetli SQL komutları enjekte etmesi işlemidir. Bu bölüm, veritabanı güvenliği projesinin en kritik ve uygulama katmanıyla en çok etkileşime giren savunma aşamasını temsil etmektedir.

4.1. SQL Injection Nedir? (Zafiyetin Mantığı)

Zafiyet, bir uygulamanın kullanıcıdan aldığı veriyi hiçbir kontrolden geçirmeden doğrudan SQL sorgusuna eklemesi durumunda ortaya çıkar.

Zafiyetli Kod Örneği:

```
-- Uygulama tarafında oluşturulan dinamik sorgu (Tehlikeli!)
USE WideWorldImporters;
DECLARE @UserName NVARCHAR(50) = '' OR '1'='1'; -- Saldırgan girişi
DECLARE @Query NVARCHAR(MAX) = 'SELECT * FROM Application.People WHERE LogonName = '' + @UserName + ''';
EXEC sp_executesql @Query;
```

Burada ' OR '1'='1' ifadesi, WHERE koşulunu mantıksal olarak her zaman doğru (True) kılar. Sonuç olarak saldırgan, kimlik bilgilerini bilmesine gerek kalmadan tüm kullanıcı listesine erişebilir.

4.2. Adım Adım SQL Injection Testi (Simülasyon)

A. Kimlik Doğrulama Atlama (Authentication Bypass)

- **Senaryo:** Application.People tablosunda yetkisiz bir şekilde kullanıcı bilgilerine erişim simüle edilmiştir.
- **Saldırı Sorgusu:**

```
-- Hacker girişi: ' OR 1=1 --
USE WideWorldImporters;
SELECT FullName, EmailAddress FROM Application.People WHERE LogonName = '' OR 1=1 --';
```

Burda - - işareti SQL'de yorum satırı anlamına gelir. Bu sayede sorgunun geri kalanındaki şifre kontrolü gibi güvenlik kısıtlamaları devre dışı bırakılır.

B. Veri Sızdırma (UNION Based SQLi)

- **Saldırı Sorgusu:**

```
-- Sistem bilgilerini çekme denemesi
USE WideWorldImporters;
SELECT FullName FROM Application.People WHERE PersonID = -1
UNION
SELECT name FROM sys.databases;
```

Burda Saldırgan, UNION operatörünü kullanarak hedef tablonun dışına çıkar ve veritabanı isimleri veya sistem tabloları gibi hassas meta verileri çalabilir.

4.3. SQL Injection'dan Korunma Yöntemleri

Veritabanının korunması kriteri doğrultusunda şu üç ana savunma yöntemi uygulanmıştır:

1. **Parametrelili Sorgular (Prepared Statements):** Kullanıcıdan gelen veri asla sorgu metniyle birleştirilmez; bir "parametre" olarak gönderilir. SQL Server bu veriyi komut olarak değil, sadece "metin" (literal) olarak algılar.
2. **Stored Procedure (Saklı Yordam) Kullanımı:** İş mantığı veritabanı içinde saklanır ve dışarıdan sadece belirli tiplerde parametre kabul edilir. Bu, saldırı yüzeyini önemli ölçüde daraltır.
3. **Input Validation (Girdi Doğrulama):** Uygulama katmanında verinin beklenen formatta (örn. sadece sayı veya tarih) olup olmadığı kontrol edilir.

4.4. Uygulamalı Savunma Analizi

Güvenli Sorgu Yapısı (Defense Mechanism)

SQL Injection saldırılarını engellemek için uygulanan parametrelili yapı şu şekildedir:

```
-- Güvenli (Parametrelili) Sorgu Örneği
USE WideWorldImporters;
DECLARE @SecureEntry NVARCHAR(MAX) = '' OR ''1''=''1'; -- Saldırı girişi

EXEC sp_executesql
    N'SELECT FullName, EmailAddress, PhoneNumber
    FROM Application.People
    WHERE LogonName = @LogonName',
    N'@LogonName NVARCHAR(MAX)',
    @LogonName = @SecureEntry;
```

Bu yapıda SQL Server, @SecureEntry içerisindeki zararlı ifadeleri bir mantıksal komut olarak değil, LogonName sütununda aranacak düz bir metin değeri olarak görür. Saldırı başarısız olur ve veri sızıntısı engellenir.

Stored Procedure ile Korunma

```
CREATE PROCEDURE dbo.usp_GetPersonProfile
    @LogonName NVARCHAR(50)
AS
BEGIN
    SET NOCOUNT ON;
    SELECT FullName, EmailAddress, PhoneNumber
    FROM Application.People
    WHERE LogonName = @LogonName;
END;
```

Stored Procedure kullanımı, dinamik SQL oluşturma zorunluluğunu ortadan kaldırırken "Type Checking" (Tip Kontrolü) yaparak güvenliği en üst seviyeye çıkarır.

5. AUDIT LOGLARI (DENETİM İZLERİ)

Veritabanı güvenliğinin sağlanması ve sürdürülebilir kılınması için sadece erişimi kısıtlamak ve verileri şifrelemek yeterli değildir; sistem üzerinde gerçekleşen tüm eylemlerin kayıt altına alınması ve izlenmesi gerekmektedir.

5.1. Denetim Mekanizmasının Amacı

SQL Server Audit özelliği, hem sunucu düzeyinde (başarısız giriş denemeleri vb.) hem de veritabanı düzeyinde (tablo sorgulamaları ve veri değişiklikleri vb.) detaylı loglama yaparak, olası güvenlik ihlallerinin tespit edilmesini ve adli analiz yapılabilmesini sağlar.

5.2. Sunucu Düzeyinde Denetim (Server Audit) Oluşturma

Denetim sürecinin ilk adımı, logların depolanacağı ortamı ve genel kuralları belirleyen ana denetim nesnesini oluşturmaktır.

Uygulama Adımları:

- **Erişim:** **Security > Audits** klasörüne sağ tıklanarak **New Audit** seçeneği ile süreç başlatılmıştır.
- **Hedef Belirleme:** Audit Destination olarak "**Application Log**" seçilmiştir. Bu sayede loglar, Windows Olay Görüntüleyicisi (Event Viewer) üzerinden de merkezi olarak izlenebilir hale getirilmiştir.
- **Aktivasyon:** Oluşturulan **Güvenlik_Denetimi** nesnesi sağ tıklanarak **Enable Audit** konumu ile faal hale getirilmiştir.

5.3. Veritabanı Düzeyinde Denetim (Database Audit Specification)

Bu aşamada, hangi spesifik tabloların veya hangi SQL komutlarının (SELECT, DELETE vb.) izleneceği belirlenir.

Uygulama Adımları:

- **Tanımlama:** **WideWorldImporters > Security > Database Audit Specifications** altından yeni bir spesifikasyon oluşturulmuştur.
- **Eylem Seçimi:** Audit Action Type olarak **SELECT** ve **DELETE** işlemleri seçilerek kritik veri hareketleri odağa alınmıştır.
- **Kapsam:** Object Class "**DATABASE**" olarak belirlenerek, tüm veritabanı genelindeki hareketler denetim altına alınmıştır.
- **Hedef Kitle:** Principal Name olarak **public** seçilerek, tüm kullanıcıların (özellikle kısıtlı yetkiye sahip kullanıcıların ve yetkisiz denemelerin) izlenmesi sağlanmıştır.

T-SQL Uygulaması:

```
-- T-SQL ile Denetim Spesifikasyonu Yapılandırması
CREATE DATABASE AUDIT SPECIFICATION [Sorgu_Takibi_Sertifikasi]
FOR SERVER AUDIT [Guvencilik_Denetimi]
ADD (SELECT ON DATABASE::[WideWorldImporters] BY [public]),
ADD (DELETE ON DATABASE::[WideWorldImporters] BY [public])
WITH (STATE = ON);
```

5.4. Denetim Kayıtlarının İzlenmesi ve Analiz

Yapılan yapılandırmanın doğruluğunu ve logların üretildiğini test etmek amacıyla SQL Server'ın yerleşik izleme araçları kullanılmıştır.

- **Log Görüntüleme:** Security > Audits > Guvencilik_Denetimi nesnesine sağ tıklanarak **View Audit Logs** seçeneği ile kayıtlar gerçek zamanlı olarak incelenmiştir.
- **Senaryo Testi:** Projenin önceki aşamasında gerçekleştirilen **SQL Injection** denemeleri sırasında oluşan anormal SELECT sorgularının; log panelinde kullanıcı adı, bilgisayar adı ve tam sorgu metni ile birlikte hatasız bir şekilde kayıt altına alındığı doğrulanmıştır.

Kurulan Audit yapısı sayesinde, veritabanı üzerinde gerçekleşen tüm kritik işlemler "**Non-repudiation**" (**İnkâr Edilemezlik**) ilkesine uygun şekilde kaydedilmektedir. Bu kayıtlar, sistemin şeffaflığını artırmanın yanı sıra, KVKK ve GDPR gibi veri koruma mevzuatlarına uyum sürecinde yasal bir kanıt niteliği taşımaktadır. Ayrıca, siber saldırı girişimlerinin erken aşamada tespit edilmesi için proaktif bir savunma hattı oluşturmaktadır.

6. SONUÇ

Proje 3: Veritabanı Güvenliği ve Erişim Kontrolü kapsamında, bir veritabanı sisteminin sadece dış tehditlere karşı değil, iç yetkilendirme hatalarına ve uygulama katmanındaki zafiyetlere karşı da nasıl korunması gerektiği kapsamlı bir şekilde simüle edilmiştir.

Bu proje, veritabanı güvenliğinin tek bir ayar değil, bir süreç olduğunu ve her katmanda (erişim, şifreleme, sorgu yapısı, denetim) ayrı ayrı önlem alınması gerektiğini ortaya koymuştur. Uygulanan bu çok katmanlı savunma stratejisi, **WideWorldImporters** veri setinin güvenli ve sürdürülebilir bir şekilde yönetilmesini mümkün kılmıştır.